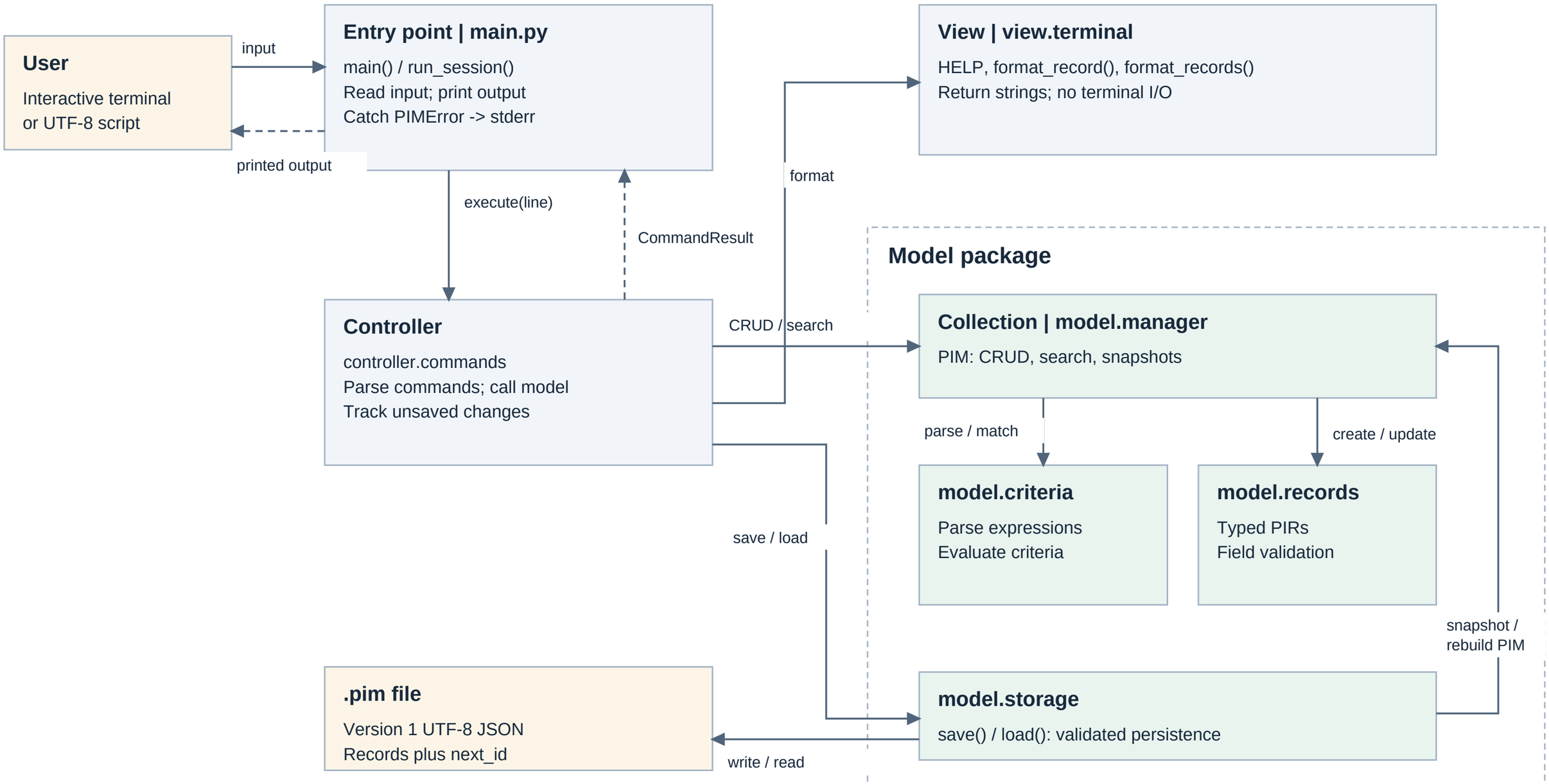
01 System architecture

MVC separation with a command-line entry point and file persistence



PIMError: ValidationError, RecordNotFound, CriteriaError, StorageError. Startup --file calls model.load() directly.

Explanation

The program uses an MVC-style separation: main.py owns terminal I/O, Controller interprets commands, the view formats records, and model owns records, validation, search and persistence. This keeps domain operations testable without terminal input. Controller requests formatted text and returns a CommandResult; main.py prints it. The model does not call the view or controller. Saving/loading is explicit: storage writes or reads the complete collection as UTF-8 JSON in a .pim file. Startup --file loading also occurs in main.py. Arrows show calls/data requests; dashed arrows show returned values or output.

02 Record and persistence classes

Typed immutable records, collection ownership and validated file snapshots

PIM | model.manager

```
# _records: dict[int, Record]
# _next_id: int
+ next_id: int {read-only property}
```

```
+ __init__()
+ create(kind: str, /, **data) -> Record
+ get(record_id: int) -> Record
+ all() -> tuple[Record, ...]
+ update(record_id: int, /, **changes) -> Record
+ delete(record_id: int) -> Record
+ search(expression: str) -> tuple[Record, ...]
+ snapshot() -> dict
+ from_snapshot(snapshot: object) -> PIM {class}
```

owns / indexes

1 — 0..*

Record | model.records {frozen dataclass}

```
+ id: int
+ kind: ClassVar[str]
```

```
+ to_dict() -> dict
+ updated(changes: dict) -> Record
```

Generated constructors take the declared fields, including id.
The subclasses inherit Record operations; kind is class-level.
All record instances are immutable after validation.

inherits

Note

```
+ text: str
+ kind = "note"
```

Task

```
+ description: str
+ deadline: datetime
+ kind = "task"
```

Event

```
+ description: str
+ start: datetime
+ alarm: datetime
+ kind = "event"
```

Contact

```
+ name: str
+ address: str
+ mobile: str
+ kind = "contact"
```

Hollow triangle: inheritance. Filled diamond: collection ownership. ValidationError guards construction and updates.

Functions | model.records

make_record(kind: str, record_id: int, data: dict) -> Record
parse_time(value: str | datetime) -> datetime
require_text(value: object, label: str) -> None
require_id(value: object) -> None

Functions | model.storage

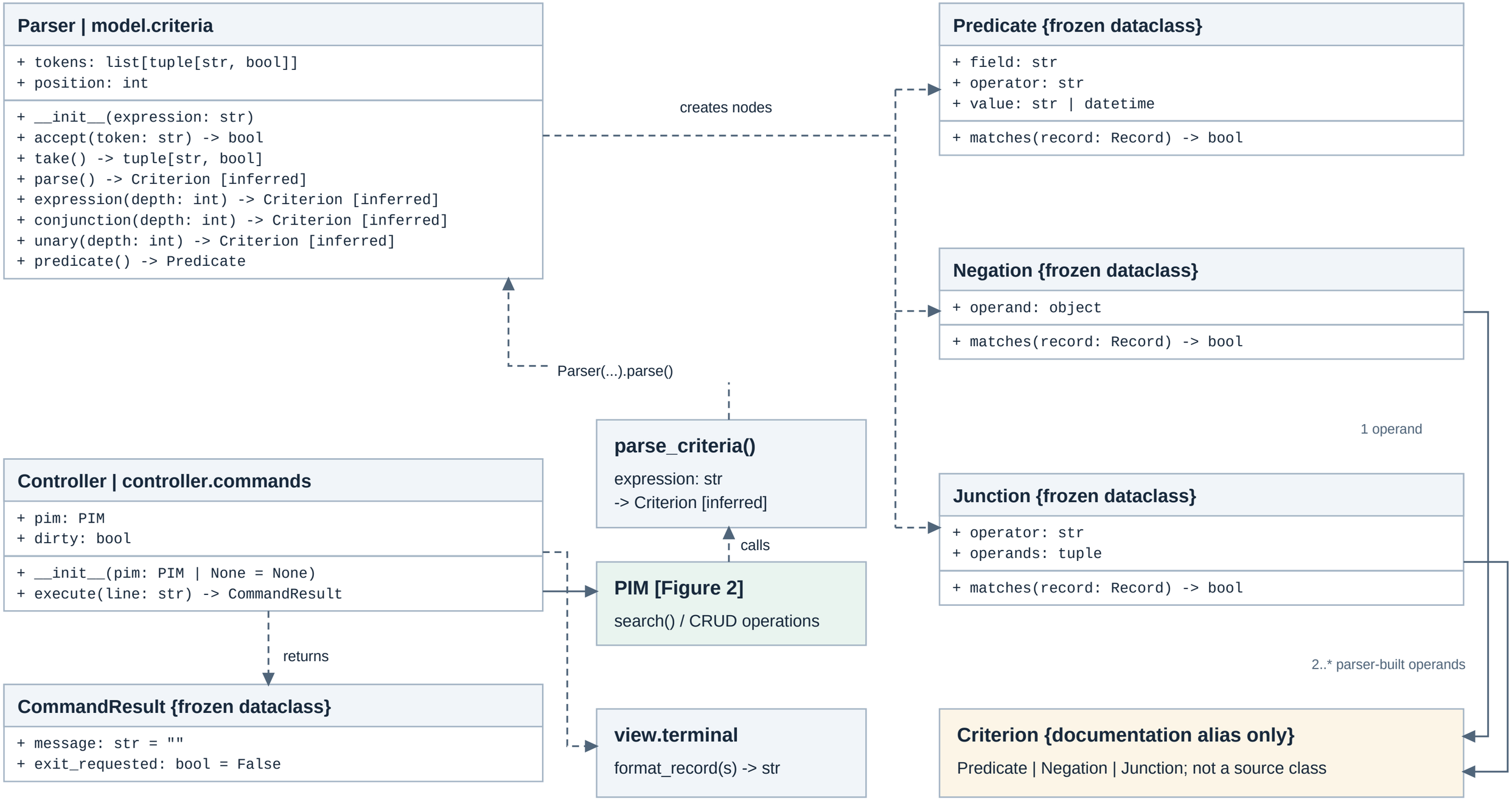
save(pim: PIM, path: str | Path) -> None
load(path: str | Path) -> PIM
pim_path(path: str | Path) -> Path
unique_object(pairs: list) -> dict

Explanation

PIM owns zero or more Record instances indexed by stable IDs. Note, Task, Event and Contact inherit id, to_dict() and updated() from Record; each fixes its class-level kind value and adds required fields. Dataclass constructors are generated from these fields. Updates use dataclasses.replace, which invokes validation before the collection replaces its reference. All times normalize to Hong Kong (+08:00); an Event additionally requires alarm <= start. Storage functions operate on PIM snapshots, with atomic replacement on save and complete reconstruction/validation on load. The composition diamond denotes collection ownership, not immediate destruction: callers can retain immutable record references.

03 Search and control classes

Expression trees, node references and command coordination



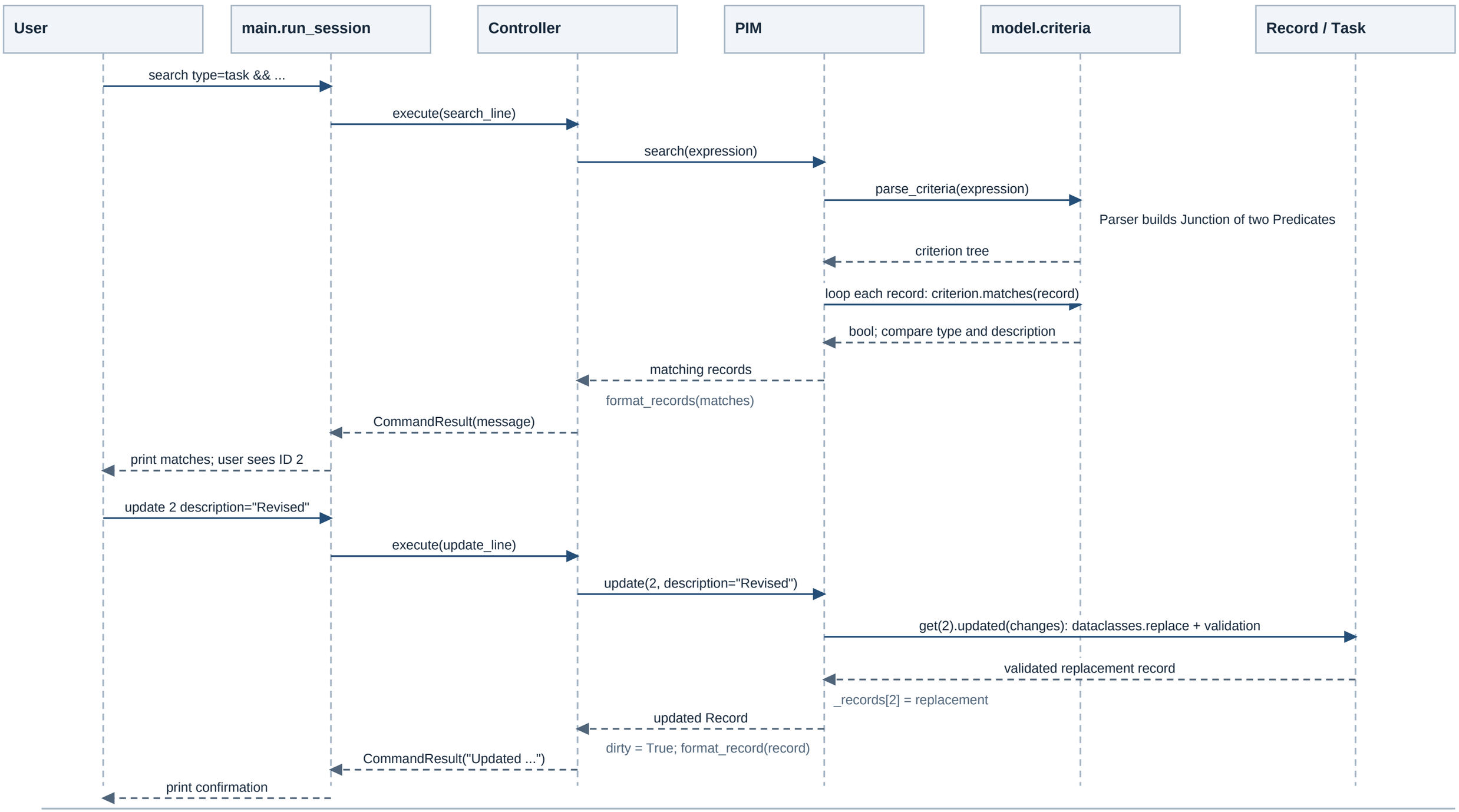
Parser / criterion return types marked [inferred] are documentation types; tokens and position are inferred from assignments.

Explanation

Parser builds Predicate, Negation and Junction objects with precedence parentheses, !, &&, then ||. Each node implements matches(record), but there is no shared interface or superclass in the Python source. Criterion is only a documentation alias for the union of those three types. Negation references one criterion; Junction references multiple criteria and short-circuits evaluation. PIM.search calls parse_criteria and evaluates the resulting tree against its records. Controller owns a PIM and an unsaved-change flag, uses view formatting functions, and returns an immutable CommandResult. Solid arrows indicate held references; dashed arrows indicate calls, object creation or returned values.

04 Search then update sequence

Example: find a task, then revise its description by the returned ID



Explanation

Assume task ID 2 already exists. The user first searches for tasks whose description contains project. The parser creates a Junction containing two Predicates; PIM evaluates it for each record and the view formats the matches. After seeing ID 2, the user issues a separate update command. PIM retrieves the record, creates a validated replacement through Record.updated, and commits it only after validation succeeds. Controller sets dirty to True and returns formatted confirmation. The sequence stops before save: the change is in memory and requires an explicit save command to persist. Expected errors are caught by run_session and shown on stderr; invalid updates leave the original record unchanged.

Design notes and API reference

These notes describe the implemented system, not a proposed redesign. The first four pages contain the four requested figures and their English explanations. This appendix supplies method/function parameters, return types and expected exceptions.

Notation and implementation details

A hollow triangle means inheritance; a filled diamond indicates collection ownership. A solid directed association represents a retained reference, while dashed dependency arrows represent calls or creation. Sequence arrows are calls/actions and dashed arrows are replies. Self is omitted from method signatures. A slash marks positional-only parameters; **data and **changes collect keyword fields. ClassVar marks class-level data. The # symbol means an underscore-prefixed attribute, protected only by convention. For **data and **changes, keyword names are the type-specific fields in Figure 2; text values are str, while time fields accept str | datetime and are stored as normalized datetime objects.

Record, Note, Task, Event, Contact, Predicate, Negation, Junction and CommandResult are frozen dataclasses. Their constructors are generated from the fields shown in Figures 2 and 3. Record and all Record subclass construction can raise ValidationError through __post_init__. Other frozen dataclasses do not independently validate constructor arguments. Parser tokens/position and return types labelled inferred are documentation types inferred from assignments or returns. Criterion abbreviates Predicate | Negation | Junction and is not defined as a class or alias in the source.

Errors and state guarantees

PIMError extends Exception. ValidationError, RecordNotFound, CriteriaError and StorageError directly extend PIMError. The CLI catches PIMError and reports the message; lower-level validation errors in file decoding are wrapped as StorageError. Validation failure during create does not consume an ID; failure during update leaves the stored record intact. Load validates a new collection before replacing the active one. Save uses os.replace after a successful temporary write. * save normally wraps OSError/ValueError as StorageError, but an OSError from the finally-block temporary-file cleanup can propagate directly; this is an implementation detail, not a guarantee that all filesystem errors are wrapped.

Sequence example and scope

Figure 4 assumes task ID 2 exists. The full search command is `search type=task && description contains "project"`. The subsequent command is `update 2 description="Revised"`. The criteria participant represents the module and its parser/evaluation objects; Record/Task is the matched task instance. Calls internal to one participant, such as Parser.parse and PIM.get, are summarized in labels. Formatting functions execute synchronously inside Controller.execute; no separate thread or GUI exists. This is the successful path. A malformed expression raises CriteriaError; a missing ID raises RecordNotFound; an invalid replacement raises ValidationError. The session reports these errors and continues. Persistence is not automatic: the example requires save to write the update to disk.

model/records.py

Signature	Responsibility	Expected exceptions / notes
parse_time(value: str | datetime) -> datetime	Parse datetime or ISO text, require date/time, normalize to +08:00.	ValidationError
require_text(value: object, label: str) -> None	Require a non-whitespace string without changing stored text.	ValidationError
require_id(value: object) -> None	Require a positive int; reject bool.	ValidationError
Record.__post_init__() -> None	Validate id/text fields and normalize datetime fields.	ValidationError
Record.to_dict() -> dict	Serialize all fields and type; times become ISO 8601 strings.	No expected domain exception for valid model inputs
Record.updated(changes: dict) -> 'Record'	Validate and return a replacement; the original is immutable.	ValidationError
Event.__post_init__() -> None	Run inherited checks and reject alarm after start.	ValidationError
make_record(kind: str, record_id: int, data: dict) -> Record	Validate record type and exact required fields, then construct it.	ValidationError

model/manager.py

Signature	Responsibility	Expected exceptions / notes
PIM.__init__() -> None [inferred]	Start an empty record collection and ID allocator.	No expected domain exception for valid model inputs
PIM.next_id() -> int	Read-only property exposing the next unused ID.	No expected domain exception for valid model inputs
PIM.create(kind: str, /, **data) -> Record	Validate data, store a record and increment next_id only on success.	ValidationError
PIM.get(record_id: int) -> Record	Validate the ID and retrieve its immutable record.	ValidationError; RecordNotFound
PIM.all() -> tuple[Record, ...]	Return records ordered by ID.	No expected domain exception for valid model inputs
PIM.update(record_id: int, /, **changes) -> Record	Retrieve, validate replacement, then replace the stored reference.	ValidationError; RecordNotFound
PIM.delete(record_id: int) -> Record	Remove one record and return it; do not reuse its ID.	ValidationError; RecordNotFound
PIM.search(expression: str) -> tuple[Record, ...]	Compile expression and evaluate it against all records.	CriteriaError
PIM.snapshot() -> dict	Return a detached versioned serializable dictionary.	No expected domain exception for valid model inputs
PIM.from_snapshot(snapshot: object) -> 'PIM'	Class method reconstructing a validated collection.	ValidationError

model/criteria.py

Signature	Responsibility	Expected exceptions / notes
Predicate.matches(record: Record) -> bool	Evaluate type equality, case-insensitive substring, or time comparison.	No expected domain exception for valid model inputs
Negation.matches(record: Record) -> bool	Invert one child result.	No expected domain exception for valid model inputs
Junction.matches(record: Record) -> bool	Evaluate child criteria with short-circuit all/any.	No expected domain exception for valid model inputs
Parser.__init__(expression: str) -> None [inferred]	Tokenize input and enforce length/quoting rules.	CriteriaError
Parser.accept(token: str) -> bool	Consume one matching unquoted token; otherwise leave position unchanged.	No expected domain exception for valid model inputs
Parser.take() -> tuple[str, bool]	Consume and return the next token.	CriteriaError
Parser.parse() -> Criterion [inferred]	Parse the full expression and reject trailing tokens.	CriteriaError
Parser.expression(depth: int) -> Criterion [inferred]	Parse disjunctions (lowest precedence).	CriteriaError
Parser.conjunction(depth: int) -> Criterion [inferred]	Parse conjunctions.	CriteriaError
Parser.unary(depth: int) -> Criterion [inferred]	Parse negation/parentheses and enforce nesting limit.	CriteriaError
Parser.predicate() -> Predicate	Validate field/operator/value and normalize time literals.	CriteriaError
parse_criteria(expression: str) -> Criterion [inferred]	Create Parser(expression), then return parse().	CriteriaError

model/storage.py

Signature	Responsibility	Expected exceptions / notes
pim_path(path: str | Path) -> Path	Expand user path and require .pim extension (case-insensitive).	StorageError
save(pim: PIM, path: str | Path) -> None	Write snapshot to a temporary file, flush/fsync, then os.replace.	StorageError*
unique_object(pairs: list) -> dict	Reject duplicate JSON keys when decoding objects.	ValidationError
load(path: str | Path) -> PIM	Decode JSON and return PIM.from_snapshot only after full validation.	StorageError

controller/commands.py

Signature	Responsibility	Expected exceptions / notes
parse_id(value: str) -> int	Convert command text to a positive integer.	ValidationError
parse_fields(arguments: list[str]) -> dict[str, str]	Parse field=value arguments and reject duplicates.	ValidationError
Controller.__init__(pim: PIM | None=None) -> None [inferred]	Use supplied PIM or create one; start with dirty=False.	No expected domain exception for valid model inputs
Controller.execute(line: str) -> CommandResult	Dispatch a command; track dirty; format results; guard load/quit.	PIMError subclasses; see called operation

view/terminal.py

Signature	Responsibility	Expected exceptions / notes
format_record(record: Record) -> str	Format every field with unambiguous JSON-quoted values.	No expected domain exception for valid model inputs
format_records(records: tuple[Record, ...]) -> str	Format the count followed by all detailed records.	No expected domain exception for valid model inputs

main.py

Signature	Responsibility	Expected exceptions / notes
run_session(controller: Controller, source: TextIO, interactive: bool) -> int	Read commands; print messages; catch PIMError; return process status.	No expected domain exception for valid model inputs
main(argv: list[str] | None=None) -> int	Parse startup options, load optional file and launch a session.	SystemExit(2) for invalid startup arguments

Source and acknowledgement

Source baseline: commit fcaee3f in Xanaxxxxxx/comp3211-pim. Figures use actual module, class and method names. This document and its diagram generation source were prepared with OpenAI Codex assistance. The group should acknowledge actual use in its Honour Declaration. The diagrams and explanations support the design-document deliverable; they do not replace the group SRS, presentation or declaration.